

Perancangan IP I2S TX AXI4-Lite pada SoC MicroBlaze V

Implementasi pada FPGA Basys3 dengan DAC PCM5102A
Termasuk Identifikasi dan Perbaikan Bug Firmware

Nama Mahasiswa
NIM

Departemen Teknik Elektro dan Teknologi Informasi
Universitas Gadjah Mada

Mei 2026

Outline

- 1 Pendahuluan
- 2 Dasar Teori
- 3 Perancangan Sistem
- 4 Implementasi Firmware
- 5 Identifikasi dan Perbaikan Bug
- 6 Hasil Pengujian
- 7 Penutup

- Sistem embedded audio membutuhkan antarmuka serial yang **stabil, presisi, dan dapat dikonfigurasi dari perangkat lunak**.
- **I2S** (Philips, 1986) adalah standar de-facto untuk transmisi audio digital antarchip: BCLK, WS/LRCK, DATA.
- **AXI4-Lite** memungkinkan prosesor mengakses register peripheral melalui model *memory-mapped I/O* yang sederhana.
- **MicroBlaze V** (RISC-V soft-core) pada ekosistem AMD/Xilinx menyediakan fondasi SoC yang terintegrasi dengan Vivado dan Vitis.
- Penelitian ini merancang IP I2S TX kustom, mengintegrasikannya ke SoC, **dan mendokumentasikan bug firmware** yang ditemukan selama validasi.

Rumusan Masalah

- 1 Bagaimana merancang IP I2S TX dengan AXI4-Lite yang dapat diakses melalui register *memory-mapped*?
- 2 Bagaimana mendukung dua mode F_s (48/96 kHz) dari satu sumber `audio_clk`?
- 3 Bagaimana memverifikasi fungsionalitas dan timing melalui ILA dan uji audio PCM5102A?

Batasan Penelitian

- TX-only, format Philips I2S, stereo.
- Bare-metal (tanpa RTOS/Linux).
- Tanpa DMA dan tanpa FIFO internal.
- Platform: Basys3 (Artix-7) + PCM5102A.
- *Programmed I/O* berbasis register murni.

Protokol I2S – Philips Standard

- **BCLK**: clock serial, *sampling* data pada tepi naik.
- **WS**: LOW = kanal kiri; HIGH = kanal kanan.
- **DATA**: serial MSB-first; satu **delay bit** (selalu 0) setelah setiap peralihan WS.
- Satu frame stereo = **64 BCLK** (32 bit kiri + 32 bit kanan).
- DATA berubah pada tepi *turun* BCLK; DAC membaca pada tepi *naik*.

[PLACEHOLDER]
Diagram timing I2S:
BCLK / WS / DATA
satu frame stereo

$$F_s = \frac{f_{\text{audio_clk}}}{\text{divider} \times 2 \times 64}$$

Mode	$f_{\text{audio_clk}}$ req.	$f_{\text{audio_clk}}$ aktual	Divider	F_s aktual
FS_MODE=0	24,576 MHz	24,597 MHz	8	48,03 kHz
FS_MODE=1	24,576 MHz	24,597 MHz	4	96,08 kHz

- FS_FAMILY=0: gunakan audio_48_clk (24,597 MHz).
- FS_FAMILY=1: gunakan audio_44_clk (22,426 MHz) untuk keluarga 44,1/88,2 kHz.

- **MicroBlaze V** sebagai AXI master.
- **AXI SmartConnect** menghubungkan prosesor ke peripheral.
- **Clocking Wizard** menghasilkan 3 clock: 100 MHz (AXI), 24,6 MHz (48k), 22,4 MHz (44,1k).
- **IP I2S** menerima data lewat register AXI4-Lite.
- Output PMOD JA → **PCM5102A**.

[PLACEHOLDER]
Screenshot Vivado Block Design:
MicroBlaze V → AXI SmartConnect
→ IP I2S → PMOD JA

5 bagian utama RTL:

- 1 **AXI4-Lite slave** – register write/read state machine.
- 2 **CDC write-counter** – sinkronisasi 2-FF dari domain AXI ke domain audio.
- 3 **BUFGMUX** – pilih audio_48_clk atau audio_44_clk.
- 4 **Clock divider** – bclk_en_w setiap 4 atau 8 siklus MCLK.
- 5 **Serializer** – Philips I2S, delay-bit, MSB-first, lebar variabel.

[PLACEHOLDER]

Blok diagram internal:

AXI Regs → CDC → BUFGMUX
→ Clock Div → Serializer → DATA

Register Map IP I2S

Offset	Nama	Fungsi
0x00	DATA_LEFT	Sampel kanal kiri; MSB di bit [SAMPLE_WIDTH-1]
0x04	DATA_RIGHT	Sampel kanal kanan; packing sama dengan kiri
0x08	CONTROL	Enable, Mute, FS_FAMILY, FS_MODE, SAMPLE_WIDTH
0x0C	RESERVED	Scratch (dipakai readback test AXI)

Bitfield CONTROL:

Bit	Field	Keterangan
0	ENABLE	1 = output aktif
1	MUTE	1 = paksa DATA = 0 (clock tetap)
2	FS_FAMILY	0 = 48/96 kHz; 1 = 44,1/88,2 kHz
3	FS_MODE	0 = Fs rendah; 1 = Fs tinggi ($\times 2$)
12:8	SAMPLE_WIDTH	Lebar bit per kanal (0 $\rightarrow$ default 32)

Constraint dan Wiring PCM5102A

Pemetaan pin (cnstr.xdc):

Sinyal	PMOD JA Pin
i2s_mclk	J1
i2s_bclk	L2
i2s_ws	J2
i2s_data	G2

[PLACEHOLDER]

Foto wiring:

Basys3 PMOD JA → PCM5102A

Semua pin LVCMOS33, cocok dengan tegangan kerja PCM5102A.

Aplikasi Vitis – Alur Utama

- 1 Tulis register CONTROL (ENABLE=1, SAMPLE_WIDTH=24, mode clock).
- 2 Loop utama: hitung sampel sinus → tulis DATA_LEFT & DATA_RIGHT → tunggu deadline.
- 3 Poll UART untuk perintah keyboard (frekuensi, kanal, mute, dll.).

Fitur yang dapat dikontrol dari terminal:

Tombol	Fungsi
1/2/5	Pilih frekuensi 1 kHz / 2 kHz / 5 kHz
L/R/B	Pilih kanal Left / Right / Both
M	Toggle MUTE
E	Toggle ENABLE
W	Siklus SAMPLE_WIDTH: 16 → 24 → 32 → 16
F/0	Toggle FS_FAMILY / FS_MODE

Gejala yang Ditemukan

#	Gejala	Kondisi	Bukan dari RTL?
1	Kedua kanal mute	Mode Both aktif	Ya
2	Kanal kanan mute	Mode Right-only	Ya
3	Amplitudo $L \neq R$	Both aktif	Ya
4	Semua frekuensi lebih rendah	1k/2k/5kHz	Ya
5	32-bit bersuara salah	SAMPLE_WIDTH=32	Ya

Temuan kunci

Seluruh 5 gejala bersumber dari **kode firmware** (helloworld.c), bukan dari RTL. RTL terbukti benar sejak awal.

Bug 1 & 2: Masker Merusak Bit Tanda Sampel

Penyebab: masker menghapus bit tanda sampel negatif setelah geseran.

Kode lama (salah):

```
// untuk 24-bit: mask = 0x0FFFFFFF
uint32_t mask = (1U << g_width) - 1U;
uint32_t samp = (uint32_t)s32 & mask;
// bit tanda [31:24] ikut terpotong!
```

Kode baru (benar):

```
// RTL hanya baca [width-1:0]
// tidak perlu masker sama sekali
int32_t s32;
if (g_width >= 32) s32 = (int32_t)s
    << 16;
else if (g_width > 16) s32 = (int32_t)s
    << (g_width-16);
else if (g_width < 16) s32 = (int32_t)s
    >> (16-g_width);
else s32 = (int32_t)s;
uint32_t samp = (uint32_t)s32; // tanpa
    mask
```

Aturan justifikasi: geser int16 sehingga MSB-nya tepat di bit [SAMPLE_WIDTH-1].

SAMPLE_WIDTH	Geseran	MSB di bit
16	0	15

Bug 3: Frekuensi Lebih Rendah dari Target

Penyebab: `usleep(periode - 2)` tidak akumulatif dan tidak presisi.

Kode lama:

```
uint32_t period_us =  
    (1000000U + sr/2U) / sr;  
usleep(period_us - 2);  
// -2 adalah tebakan  
// error terakumulasi tiap iterasi  
// di 96kHz: 10us - 2 = 8us      20% off
```

Kode baru (deadline absolut):

```
// rdcycle64(): baca CSR mcycle RISC-V  
// resolusi 10 ns pada 100 MHz CPU  
uint64_t deadline = rdcycle64();  
while (1) {  
    uint64_t period =  
        100000000ULL / current_sample_rate();  
    stream_sample();  
    poll_uart();  
    deadline += period; // maju tepat 1  
        periode  
    while (rdcycle64() < deadline) {}  
}
```

Keunggulan deadline-based

Overhead loop (AXI write, UART poll) dikompensasi otomatis. Presisi setara osilator 100 MHz board – tidak ada drift kumulatif.

Perbandingan Metode Timing Loop

Properti	usleep() lama	rdcycle64() baru
Resolusi waktu	$\approx 1 \mu\text{s}$	10 ns
Kompensasi overhead loop	Tebakan ($-2 \mu\text{s}$)	Otomatis
Akumulasi error	Ya	Tidak
Bekerja di 96 kHz ($10,4 \mu\text{s}/\text{sampel}$)	Tidak ($\approx 20\% \text{ off}$)	Ya
Header tambahan dibutuhkan	Tidak	Tidak

- `rdcycle/rdcycleh` adalah instruksi CSR standar RISC-V, tersedia pada MicroBlaze V tanpa header khusus.
- Loop ganda (*read-hi, read-lo, read-hi*) mencegah race condition saat carry dari bit 31 ke bit 32.

Yang dikonfirmasi:

- 64 BCLK per frame stereo
- WS beralih 1 BCLK *sebelum* MSB
- Bit pertama setelah WS selalu 0 (delay bit)
- BCLK = 3,0746 MHz
- WS = 48,03 kHz
- Kedua kanal L dan R aktif

[PLACEHOLDER]
Tangkapan ILA:
BCLK / WS / DATA
(trigger: tepi turun WS)

Hasil Verifikasi Firmware Setelah Perbaikan

Skenario Uji	Hasil yang Diharapkan	Status
Both + 1 kHz	L dan R identik, 1 kHz	✓
Left only	L aktif, R = 0	✓
Right only	R aktif, L = 0	✓
Both + 2 kHz	L dan R identik, 2 kHz	✓
Both + 5 kHz	L dan R identik, 5 kHz	✓
MUTE aktif	Diam, BCLK/WS tetap	✓
WIDTH = 16 bit	Amplitudo benar	✓
WIDTH = 32 bit	Amplitudo penuh, benar	✓

- Sinyal sinus 1 kHz, 2 kHz, dan 5 kHz terdengar jelas melalui speaker/headphone.
- Mode Both: L dan R **amplitudo identik**.
- Mode Left/Right: kanal inaktif benar-benar senyap.
- Mode Mute: tidak ada suara, BCLK/WS tetap berjalan (DAC tetap terkunci pada clock).

[PLACEHOLDER]

Osiloskop/Audacity setelah fix:
kanal L = R, amplitudo identik

Ringkasan Analisis Frekuensi

Besaran	Target nominal	Hasil (clock aktual)	Selisih
$f_{\text{audio_clk}}$ 48k	24,576 MHz	24,597 MHz	+0,09%
BCLK (FS_MODE=0)	3,072 MHz	3,0746 MHz	+0,08%
F_s (FS_MODE=0)	48,000 kHz	48,03 kHz	+0,06%
BCLK (FS_MODE=1)	6,144 MHz	6,1493 MHz	+0,08%
F_s (FS_MODE=1)	96,000 kHz	96,08 kHz	+0,08%

- Selisih $<0,1\%$ – berasal dari keterbatasan rasio MMCM Artix-7, bukan dari bug.
- Kedua mode F_s tervalidasi setelah bug firmware timing diperbaiki.

- ❶ **IP I2S TX AXI4-Lite** berhasil dirancang dan diintegrasikan pada SoC MicroBlaze V di Basys3 dengan register map aktif (0x00, 0x04, 0x08).
- ❷ **Dua mode F_s** (48,03 kHz dan 96,08 kHz) berhasil dihasilkan dari satu audio_48_clk (24,597 MHz) melalui divider 8 dan 4.
- ❸ **Timing I2S** terverifikasi via ILA: 64 BCLK/frame, delay-bit wajib, WS sesuai spesifikasi Philips.
- ❹ **Tiga bug firmware** ditemukan dan diperbaiki *sepenuhnya di sisi software* – RTL tidak dimodifikasi:
 - Masker merusak bit tanda sampel negatif.
 - Justifikasi sampel 32-bit tidak tepat (MSB salah posisi).
 - Loop timing usleep tidak akumulatif; diganti rdcycle64() dengan presisi 10 ns.
- ❺ **Validasi audio PCM5102A** berhasil: L = R amplitudo identik, seleksi kanal berfungsi, frekuensi 1/2/5 kHz terukur benar.

Saran Pengembangan

- ➊ Menambahkan **FIFO TX** dan **watermark interrupt** untuk mengurangi beban CPU dan mencegah *underrun* pada beban tinggi.
- ➋ Mengintegrasikan **DMA** untuk *streaming* audio kontinu tanpa keterlibatan CPU per sampel.
- ➌ Menambahkan register **STATUS** (level buffer, flag underrun) untuk mendukung pengembangan driver berbasis OS.
- ➍ Mengembangkan mode **RX** atau ekstensi **TDM/multi-channel** untuk sistem akuisisi audio yang lebih kompleks.
- ➎ Karakterisasi akurasi clock pada berbagai suhu dan kondisi beban untuk aplikasi yang membutuhkan presisi tinggi.

Terima Kasih

Kontribusi Utama

- IP core I2S TX AXI4-Lite berbasis Verilog untuk MicroBlaze V
- Alur lengkap: RTL → Block Design → Vitis → Validasi hardware
- Dokumentasi dan perbaikan 3 bug firmware dengan analisis akar penyebab

File RTL	<code>i2s_slave_lite_v1_0_S00_AXI.v</code>
Firmware	<code>micblaze/ws/rtlfix/src/helloworld.c</code>
Laporan bug	<code>thesi2s/BUGFIX_REPORT.md</code>